

Comparative Study: Technology Choices for R-Type

This study analyzes and justifies the main technological choices for the r-type project: an open-source game written from the ground up in C++ based on classic game R-Type. This comparative study aims to find the best fitted librairies and technologies to build such a project and make it as maintainable as possible.

Contents

1. Introduction	3
2. Graphics & Multimedia	3
2.1. SDL3	3
2.2. Alternatives	3
2.3. Comparative overview	3
2.4. Graphic & Multimedia choice rationale	3
3. Scripting & Embedding	3
3.1. Lua	3
3.2. sol2 (C++ bindings for Lua)	4
3.3. Alternatives	4
3.4. Comparative overview	4
3.5. Scripting language choice rationale	4
4. Dependency & Build Management	4
4.1. CPM.cmake	4
4.2. Alternatives	4
4.3. Comparative overview	4
4.4. Dependency & Build Management system choice rationale	5
5. Utility Libraries	5
5.1. Boost	5
5.2. Alternatives	5
5.3. Utility library choice rationale	5
6. Conclusion	5

1. Introduction

Selecting technology for a C++, cross-platform, real-time application requires balancing control and development velocity. This study presents a comparative rationale for:

- SDL3 as a quick prototyping graphic library which does not sacrifice cross-platform capabilities.
- RayLib as an durable and well fitted graphic library for 2D cross-platform games.
- Lua as the embedded scripting language, bound via sol2 to modern C++.
- CPM.cmake for dependency and build management within pure CMake workflows.
- Boost for high-quality, peer-reviewed utility libraries beyond the standard library.

2. Graphics & Multimedia

2.1. SDL3

- Cross-platform coverage (Windows, Linux, even mobile if needed).
- Unified handling of windowing, input, audio, timers, events.
- Stable C API that interoperates cleanly with C++ and custom renderers.
- Library already well understood by a large part of the team.
- Prototyping speed quite reduced compared to libraries with narrower usage context.

2.2. Alternatives

- SFML: Friendly API and quick onboarding; smaller ecosystem, less low-level configurability.
- Allegro: Solid 2D focus; comparatively smaller community and slower extension cadence; no previous knowledge of it in the team.
- Raw APIs (OpenGL, Vulkan, DirectX): Maximum control, but higher complexity and reduced portability without an additional abstraction layer; typically paired with SDL for platform handling.
- RayLib: Well suited API for 2D/3D games, low complexity with low complexity cost; but library not understood by any team member so it will need a long time of acclimatation.

2.3. Comparative overview

Library	Platform support	API Complexity	Abstraction depth	Ecosystem	Prototyping speed
SDL3	Excellent	Medium	High	Large	Medium
SFML	Good	Low	Medium	Medium	High
Allegro	Good	Medium	Medium	Small	Medium
GL/VK/DX	Variable	High	Low	Large	Low
RayLib	Good	Low	High	Medium but well suited	Medium

2.4. Graphic & Multimedia choice rationale

Due to our previous knowledge of the SDL3 library we choose to use-it for our prototype to ensure us a quick and rapidly advancing prototyping phase. We can also profit of its deep system access to test graphic idea and promote creativity in the team. In the second part of our project we will migrate our rendering system to the Raylib to take advantage of its well suited API for 2D games and its already provided easy integration with ECS-like Data Driven Models.

3. Scripting & Embedding

3.1. Lua

- Lightweight VM with fast startup and low memory profile.
- Predictable performance suitable for real-time systems.
- Simple syntax with a gentle learning curve for designers and modders.

- Proven track record in games and embedded runtime customization.

3.2. sol2 (C++ bindings for Lua)

- Modern, header-only C++11 binding layer.
- Type-safe conversions, exception safety options, and minimal overhead abstractions.
- Straightforward CMake integration

3.3. Alternatives

- Python: Rich ecosystem but heavier footprint and slower embed/startup characteristics for real-time loops.
- JavaScript engines (V8, Duktape, QuickJS): Flexible, but typically heavier and more complex to embed and tune for low-latency workloads.
- AngelScript/Squirrel: Game-oriented designs; smaller communities and fewer modern C++ binding options than Lua+sol2.

3.4. Comparative overview

Library	Runtime footprint	Performance	C++ bindings	Community	Ease of use
Lua	Low	High	sol2 (modern)	Large	Easy
Python	High	Medium	pybind11 (C-API)	Largest	Easy
JavaScript	Medium	Medium	engine-specific	Largest	Medium
AngelScript/ Squirrel	Low	High	native	Small	Medium
WebASM	Low	High	High	Small	Hard

3.5. Scripting language choice rationale

Lua with sol2 provides a tight, low-overhead bridge between C++ and gameplay logic, enabling rapid iteration and modding without compromising frame-time budgets.

4. Dependency & Build Management

4.1. CPM.cmake

- CMake-native dependency retrieval atop FetchContent with a concise, declarative interface.
- No external package manager installation required (smoother onboarding and CI).
- Reproducible by commit/tag pinning; good cache behavior in CI.
- Works well for projects that already standardize on CMake.

4.2. Alternatives

- vcpkg: Robust, curated ports and binary caching; requires separate tool bootstrap and environment alignment.
- Conan: Very flexible and enterprise-ready; more configuration overhead and external tooling.

4.3. Comparative overview

Library	Overhead	CMake integration	Simplicity
CPM.cmake	Low	Excellent	High
vcpkg	Medium	Good	Medium
Conan	High	Good	Medium

4.4. Dependency & Build Management system choice rationale

CPM.cmake keeps the workflow self-contained in CMake, minimizing external dependencies while retaining reproducibility and CI efficiency.

5. Utility Libraries

5.1. Boost

- Broad, peer-reviewed collection complementing the standard library (e.g., smart pointers, containers, algorithms, I/O, parsing).
- High-quality implementations that often inform future C++ standards.
- Strong portability guarantees and active maintenance.

5.2. Alternatives

- C++ Standard Library only: Stable baseline, but missing advanced facilities (e.g., robust parsing, extended containers, asynchrony patterns pre-C++23, etc.).
- Other utility libraries: Often narrower in scope, less standardized, or less aligned with modern C++ idioms.

5.3. Utility library choice rationale

Boost augments the standard library with well-tested components that reduce time-to-solution while preserving performance and code quality. It also provides network API that are safe even cross-platform and are standardized for network transactions (endianness compensated for)

6. Conclusion

The selection of SDL3, Lua with sol2, CPM.cmake, and Boost balances performance, portability, and developer productivity for the current game prototype:

- SDL3: Cross-platform multimedia foundation with a clean path to custom rendering.
- Lua + sol2: Lightweight, high-performance scripting and safe C++ interop.
- CPM.cmake: Simple, reproducible, CMake-native dependency management.
- Boost: Extensive, reliable utilities that accelerate development beyond the STL.

These technologies align with our goals of maintainability, rapid iteration, and cross-platform deployment. We also keep a record of what should be done when we migrate over to RayLib.